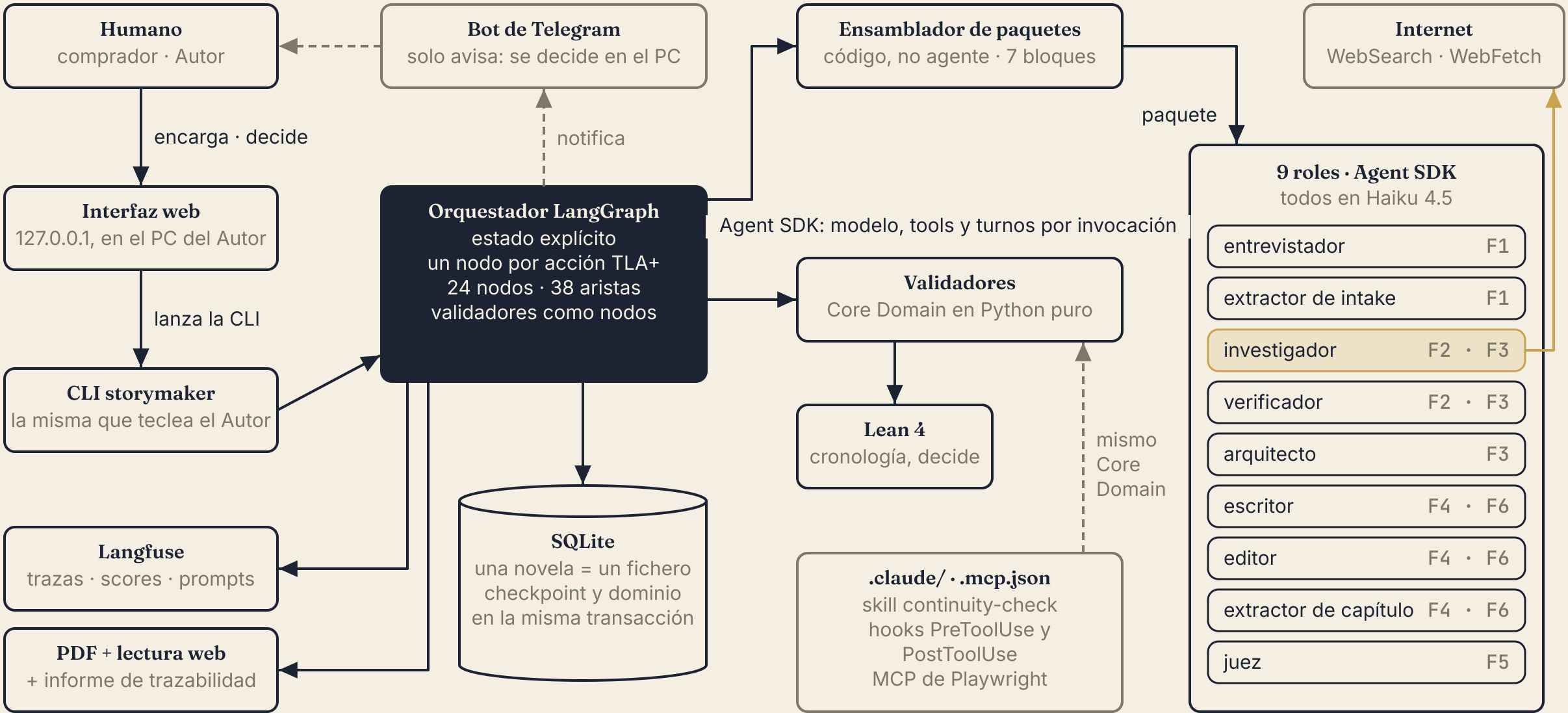


Un orquestador sin memoria propia: todo vive en el SQLite de cada novela



Dieciocho decisiones fijadas gobiernan el resto del sistema

Decisión	Elegida	Consecuencia principal
Dominio	Novela histórica personalizada	La investigación y la validación formal tienen materia real que verificar.
Motor de orquestación	LangGraph con estado explícito	Un nodo por acción de la especificación TLA+, con el mismo nombre: la correspondencia es literal.
Agentes	Claude Agent SDK (Python), todos en Haiku 4.5	Modelo, herramientas y turnos se fijan por invocación; internet entra por WebSearch y WebFetch.
Roles	Nueve, con funciones separadas	El editor repara dentro del bucle, el juez mide fuera, el verificador comprueba el corpus y los extractores miden lo que no puede declarar quien lo produjo.
Validadores	Nodos del grafo, nunca herramientas de un agente	Ningún modelo puede saltarse una validación: las aristas condicionales leen booleanos de Python.
Paso de contexto	Empuje determinista en siete bloques	El escritor no tiene herramientas de recuperación: contexto acotado, reproducible y auditable.
Estado	Un único fichero SQLite por novela	Checkpoint del grafo y datos de dominio en la misma transacción: no pueden desincronizarse.
Versiones	Capítulos inmutables + manifiesto	Conservar la versión anterior es una propiedad de la estructura, no una disciplina.
Ejecuciones de fase	Grafo de fase_run inmutables	Rehacer, reanudar, ramificar y regenerar son la misma operación con distinto punto de entrada.

El humano decide en su PC, y el límite de contexto se garantiza sumando techos

Decisión	Elegida	Consecuencia principal
Intervención humana	Cinco gates bloqueantes; Telegram solo avisa	La máquina duerme en disco; nada fuera del PC del Autor reanuda: la API solo escucha en 127.0.0.1. Desactivables en batch.
Operación del arnés	La interfaz lanza la CLI como proceso aparte	El servidor no guarda nada en memoria: reiniciarlo no mata ninguna ejecución.
Validación formal	Lean 4 (la historia) + TLA+ (el arnés)	Lean verifica la cronología concreta; TLC, el comportamiento del sistema.
Observabilidad	Langfuse, spans manuales + OTLP opcional	La semántica la pone el orquestador: sesión = novela, span = capítulo, rol e intento.
Presupuesto de contexto	100.000 tokens concurrentes, por construcción	No se puede medir en vivo, así que se acota a priori (§12).
Pila	FastAPI <i>package by feature</i> + React y Vite en FSD v2.1	En el backend, una fase es una carpeta.
Embeddings	FastEmbed local, MiniLM-L12 multilingüe, 384d, <code>sqlite-vec</code>	Deciden qué material entra en cada paquete; locales, sin red y deterministas.
Correspondencia documento→código	Tests de trazabilidad en G1	Un apartado sin implementar o un validador que deja de bloquear se ven en cuanto corre G1.
Requisitos de la spec	REQ-BE-nn y REQ-FE-nn derivados del documento	La spec se lee como una lista de compromisos comprobables.

La decisión de la que cuelga todo: los validadores son nodos del grafo, no herramientas que un agente elige llamar. El editor recibe el informe de incidencias ya producido y solo emite el parche; las transiciones dependen de valores de Python, que es lo que TLC puede verificar.

Quien escribe no aprueba: el escritor no valida, el editor no puntúa, el juez no edita

Rol	Fase	Entrada	Salida	Herramientas
entrevistador	1	Premisa libre, respuestas, texto pegado	Brief Pydantic	—
extractor de intake	1	Texto pegado en cuarentena	Filas tipadas de intake_dato	—
investigador	2 · 3	Período y lugar del brief; en Plotting, un hueco	mundo_hecho + mundo_fuente, o no_encontrado	WebSearch, WebFetch: la única salida a la red
verificador	2 · 3	Pares enunciado–cita (≤ 300 caracteres)	Veredicto de respaldo por hecho	—
arquitecto	3	Brief + corpus	Premisa, tema, canon, escaleta	—*
escritor	4 · 6	Paquete de contexto de 7 bloques	Prosa del capítulo	—
editor	4 · 6	Capítulo + informe de incidencias	Parche de corrección	—
extractor de capítulo	4 · 6	Capítulo + escaleta de sus escenas	Resumen, continuidad, usos, cronología, ejecución	—
juez	5	Novela completa + rúbrica	Ocho notas 1–10 con justificación	—

El contrato viaja con la llamada. Cada salida es un modelo Pydantic; su JSON Schema va en el prompt y schema_guard valida contra el mismo modelo.

Los extractores son roles porque consumen contexto y necesitan techo, y porque miden lo que no puede declarar quien lo produjo.

* El código concede al arquitecto dos herramientas propias sin red, sumar_días y edad_en_fecha, que §5 no lista.

El escritor no busca su contexto: lo recibe en siete bloques y 12.000 tokens

1	Encargo	800	#	Bloque	Contenido	Techo
			1	Encargo	Capítulo N: escenas, beats, objetivo, extensión, hitos de arco y lo pendiente de N-1	800
2	Canon relevante	2.500	2	Canon relevante	Fichas de personajes y escenarios presentes, más los vecinos semánticos	2.500
			3	Continuidad	Estado al cierre de N-1 y lo que ya ha pasado, un evento por línea	2.500
3	Continuidad	2.500	4	Memoria	Texto íntegro de N-1 + resúmenes previos por similitud	3.000
			5	Anclajes	Hechos anclados y vecinos del corpus sellado, con firmeza y fuente	1.500
			6	Reglas	Voz, estilo, glosario, prohibidas, licencia; repeticiones y cierres ya gastados	1.200
			7	Personalización	Elementos del brief que este capítulo tiene que tocar	500
			Total			12.000
4	Memoria	3.000	Relevancia, no recencia. Los bloques 2, 4 y 5 se llenan con una consulta derivada de la escaleta, vectorizada con FastEmbed y resuelta por KNN exhaustivo en <code>sqlite-vec</code>. La hace el ensamblador, no el agente. Orden de recorte: 4 · 2 · 5 · 7 · 6 · 1 · 3. La memoria se recorta la primera; la continuidad, la última. Cada paquete se persiste en <code>paquete_contexto</code> y se enlaza desde su span.			
5	Anclajes	1.500				
6	Reglas	1.200				
	Personalización	500				
	Total	12.000				

El límite de 100.000 tokens no se vigila: se garantiza con una suma de techos

Perfil	Prompt + skills	Contexto	Salida		Techo sobre 100.000	Cuota de herramientas	Turnos
entrevistador	2.000	4.000	2.000		8.000	—	1
extractor de intake	1.500	3.000	1.500		6.000	—	1
investigador · sesión inicial	2.000	31.000	6.000		45.000	WebSearch ×3 · WebFetch ×3	20
investigador · micro (Plotting)	2.000	11.000	1.000		14.000	WebSearch ×1 · WebFetch ×1	2
investigador · dirigido (exhaustivo)	2.000	11.000	2.000		15.000	WebSearch ×1 · WebFetch ×1	4
verificador	2.000	8.000	2.000		12.000	—	1
arquitecto	2.000	15.000	8.000		25.000	sumar_dias ×4 · edad_en_fecha ×4	6
escritor	5.000	12.000	3.000		20.000	—	1
editor	5.000	12.000	3.000		20.000	—	1
extractor de capítulo	2.000	8.000	2.000		12.000	—	1
juez	3.500	26.000	3.000		32.500	—	1

Peor caso del sistema: 45.000 tokens, la sesión inicial del investigador, con tres **WebFetch** acotados a 10.000. El grafo es secuencial: hay un solo agente abierto a la vez.

Impuesto por el arnés. Un hook `PreToolUse` deniega la llamada que excede la cuota; un hook `PostToolUse` recorta la respuesta con `updatedToolOutput`. Una guarda rechaza el prompt que excede su techo.